

Structured Outputs in GenAI

Why Plain Text Is Not Enough

Most GenAI applications begin with a very simple flow:

```
User
↓
LLM
↓
Text Response
```

For example:

```
String response = chatClient.prompt()
    .user("Explain Docker")
    .call()
    .content();
```

The response may look like:

```
Docker is a platform that allows developers to package
applications into containers...
```

For a chatbot, this works perfectly because the final consumer of the response is a **human**.

Humans are very good at understanding unstructured information.

Consider this sentence:

```
Rahul wants a meeting tomorrow at 4 PM for 30 minutes
to discuss Docker deployment.
```

A human can immediately identify:

```
Person    → Rahul
Date      → Tomorrow
Time      → 4 PM
Duration  → 30 minutes
Topic     → Docker deployment
```

But a Java application does not automatically understand those individual pieces of information.

From the application's perspective, the entire response is simply:

```
String response;
```

If the application later wants to perform operations such as:

```
meeting.getTime();
```

or:

```
if (meeting.getDuration() > 60) {
    // some logic
}
```

or eventually:

```
calendar.createEvent(meeting);
```

the information needs to exist in a predictable structure.

Human-Readable vs Machine-Usable Data

Consider this response:

```
The meeting with Rahul is scheduled for tomorrow at
```

4 PM and should last for approximately 30 minutes.

A human can easily understand it.

Software works much better with something like:

```
String attendee = "Rahul";  
String date = "2026-10-02";  
String time = "16:00";  
int duration = 30;
```

Now each value can directly participate in application logic.

The fundamental distinction is:

Humans prefer meaning.

Software prefers structure.

LLMs naturally generate language.

Applications usually work with structured values such as:

```
Objects  
Arrays  
Numbers  
Booleans  
Enums  
Dates  
IDs
```

When we integrate an LLM into a real application, we often need a bridge between these two worlds.

```
Natural Language  
↓  
LLM  
↓  
Structured Data
```

↓
Application Logic

That bridge is **Structured Output**.

What Is Structured Output?

Structured output means asking the model to produce information in a **predefined structure that our application can reliably consume**.

To understand this, consider a normal REST API.

Instead of returning:

```
The user's name is Rahul.  
He is 28 years old and his email is rahul@gmail.com.
```

we normally return:

```
{  
  "name": "Rahul",  
  "age": 28,  
  "email": "rahul@gmail.com"  
}
```

The second response follows a structure.

We have effectively defined a **contract** for the response.

The same idea can be applied to an LLM.

Without structured output:

```
Prompt  
↓  
LLM  
↓  
Free-form Text
```

With structured output:

```
Prompt
↓
LLM
↓
Expected Schema
↓
Structured Data
```

Example: Meeting Request

Suppose we are building a small meeting scheduling application.

We want the LLM to extract:

```
MeetingRequest

title          → String
attendee       → String
date           → String
time           → String
durationMinutes → Integer
type           → MeetingType
```

Instead of producing:

```
Sure! I've extracted the meeting information.
Rahul should attend the Docker meeting tomorrow at 4 PM...
```

we want the response to represent something like:

```
{
  "title": "Docker Deployment Discussion",
  "attendee": "Rahul",
  "date": "2026-10-02",
  "time": "16:00",
  "durationMinutes": 30,
  "type": "ONE_ON_ONE"
}
```

Our application can then work with:

```
MeetingRequest meeting;
```

and access individual values directly:

```
meeting.attendee();  
meeting.date();  
meeting.time();  
meeting.durationMinutes();
```

This is what makes structured output useful.

The model's response is no longer just something we display to the user.

It becomes **data that participates in our application logic**.

Structured Output Is Not Just JSON

A common misunderstanding is:

| Structured output means making the LLM return JSON.

JSON is a common representation, but it is not the core idea.

The deeper concept is:

| The model's output follows a predefined structure understood by our application.

For example, in Java we may ultimately want:

```
public record MeetingRequest(  
    String title,  
    String attendee,  
    String date,  
    String time,  
    int durationMinutes  
) {}
```

Conceptually:

```
LLM
↓
Structured Representation
↓
MeetingRequest Object
```

JSON may exist somewhere in between, but the Java application ultimately wants an **object**.

What Is a Schema?

A schema defines what our structured data should look like.

For example:

```
MeetingRequest
```

may contain:

```
title           → text
attendee        → text
date            → text
time            → text
durationMinutes → number
```

A simple way to understand a schema is to think of a form.

Without a schema:

```
Tell me about yourself.
```

The person can write anything.

That is similar to free-form output.

With a form:

Name: _____

Age: _____

Email: _____

the expected structure has already been defined.

The person still provides the values:

Name → Rahul
Age → 28
Email → rahul@gmail.com

but the available fields are predetermined.

Structured output works on the same principle.

We define the slots.

The LLM fills the values.

Why Structured Output Matters in Real Applications

Once the LLM response becomes a proper application object, normal programming becomes possible.

For example:

```
if (meeting.type() == MeetingType.INTERVIEW) {  
    sendInterviewInstructions();  
}
```

or:

```
if (meeting.durationMinutes() > 60) {  
    requireApproval();  
}
```

```
}
```

or eventually:

```
googleCalendar.createEvent(meeting);
```

At this point, the LLM is no longer functioning only as a chatbot.

It has become one component inside a larger software system.

Why Not Just Prompt the LLM to Return JSON?

A natural question is:

Why do we need structured output?

Can't we simply tell the model to return JSON?

For small experiments, this can work.

For example:

```
Extract the meeting information.  
  
Return JSON in the following format:  
  
{  
  "attendee": "",  
  "date": "",  
  "time": ""  
}
```

The model may return:

```
{  
  "attendee": "Rahul",  
  "date": "2026-10-02",
```

```
"time": "16:00"  
}
```

For a demo, this may be sufficient.

The problem becomes more important when moving from:

Demo

to:

Software

LLMs generate responses probabilistically.

Even after being asked to return JSON, the model could respond with:

Sure! Here's the JSON you requested:

```
{  
  "attendee": "Rahul",  
  "date": "2026-10-02",  
  "time": "16:00"  
}
```

A human can easily ignore the first sentence.

A JSON parser may not.

Prompt Instruction vs Structured Contract

This is the key distinction.

When we say:

Please return JSON.

we are giving the model an **instruction**.

Conceptually:

```
"Please behave like this."
```

Structured output instead focuses on defining what the application expects.

Conceptually:

```
"This is the structure the application expects."
```

Instead of relying only on:

```
LLM
↓
Hopefully valid JSON
```

we move toward:

```
LLM
↓
Defined Schema
↓
Expected Structured Output
↓
Application Object
```

This is much closer to how normal software systems are designed.

Structured Output vs Tool Calling

Structured output and tool calling can look similar because both may involve structured data.

But they solve different problems.

Structured Output

Structured output is about the **format of the model's final result**.

Example:

```
User
↓
LLM
↓
MeetingRequest
```

The result may represent:

```
{
  "title": "Docker Discussion",
  "attendee": "Rahul",
  "date": "2026-10-02",
  "time": "16:00",
  "durationMinutes": 30
}
```

Nothing has been executed.

The structured object itself is the result we wanted.

Tool Calling

Tool calling is about the model deciding to invoke an external function or action.

Suppose the user asks:

```
What is the weather in Delhi?
```

The flow could be:

```
User
↓
LLM
↓
Decides to call Weather Tool
↓
Tool Arguments
```

```
↓  
Weather API  
↓  
Tool Result  
↓  
LLM  
↓  
Final Response
```

The model may generate arguments such as:

```
{  
  "city": "Delhi"  
}
```

This data is structured, but its purpose is different.

It exists so that a **tool can be executed**.

The Core Difference

Structured Output

LLM produces structured data as the final result.

Tool Calling

LLM produces structured arguments so that an external function or action can be executed.

In our meeting example:

```
User:  
Schedule a Docker discussion with Rahul tomorrow  
at 4 PM for 30 minutes.
```

Structured output gives us:

```
MeetingRequest meeting;
```

At this stage, nothing has been scheduled.

We have only converted:

```
Human Language
  ↓
Application Data
```

Where MCP Comes In

After extracting the meeting information, one important piece is still missing:

```
How does our application communicate with Google Calendar?
```

At the moment, our flow is only:

```
User
  ↓
LLM
  ↓
MeetingRequest
```

We have created structured data, but no external action has happened.

The next step could extend the application to something like:

```
User
  ↓
LLM
  ↓
MeetingRequest
  ↓
External Calendar System
  ↓
Calendar Event Created
```

This is where concepts such as **tools, external integrations, and MCP** become relevant.

The structured output tells our application **what the user wants**.

The external integration is responsible for **actually doing it**.

Final Mental Model

Keep these three ideas separate:

Plain Text

LLM generates information for a human to read.

Structured Output

LLM converts information into a predictable structure that software can use.

Tool Calling / External Integration

The application or model uses that structured information to perform an external action.

For our meeting application:

"Meet Rahul tomorrow at 4 PM for 30 minutes"

↓

LLM

↓

MeetingRequest

↓

Application can use the data

↓

Calendar integration later

Structured output is therefore an important step in moving from:

AI that talks

toward:

AI that participates in software systems